Rapport de code

SciPy

KOLA MARCEL

Analyse Numérique

11 avril 2026

I. Explication de code avec SciPy

1. on a l'initialisation des variable avec :

```
x_pts = [0, 1, 2, 3, 4]
y_pts = [1, 3, 2, 5, 4]
valeur_test = 2.5
```

2. **interp1d** calcule et stocke en mémoire les coefficients de chaque polynôme du second degré (un par intervalle), mais ne les affiche pas. C'est une boîte noire numérique.

```
spline_scipy = interp1d(x_pts, y_pts, kind='quadratic')
```

3. **Évaluation locale :**On appelle la spline comme une fonction : elle identifie dans quel intervalle se trouve x=2.5 (ici [2,3]), puis évalue le polynôme correspondant. Résultat : un nombre décimal.

```
y_test_scipy = spline_scipy(valeur_test)
```

4. **Évaluation globale (pour le graphe) :** On génère 100 points régulièrement espacés et on évalue la spline sur tous ces points pour tracer une courbe lisse.

```
x_fine = np.linspace(min(x_pts), max(x_pts), 100)
```

II. Guide Technique : Interpolation et Évaluation avec SciPy

1. Construction

interp1d(x_pts, y_pts, kind='quadratic') : Elle crée une structure de données contenant les coefficients de chaque intervalle. La « construction » consiste ici à calculer la matrice des coefficients de splines.

L'Algorithme interne : Solveur de Matrice Bande (Banded Matrix Solver)

L'objectif est la performance mémoire et CPU.

1. **Allocation** : SciPy alloue des tableaux NumPy en mémoire (souvent en appelant des bibliothèques bas niveau en Fortran comme FITPACK).
2. **Remplissage matriciel** : L'algorithme remplit une « matrice bande » (une matrice contenant beaucoup de zéros, avec les valeurs concentrées sur la diagonale) représentant les conditions de la spline quadratique.
3. **Résolution numérique** : Il utilise un algorithme de décomposition LU optimisé pour les matrices bandes afin de trouver les coefficients flottants des polynômes.
4. **Sortie** : Il ne retourne pas d'équation, mais un « objet » Python en mémoire vive contenant simplement deux listes : les limites des intervalles (x_pts) et la matrice de tous les coefficients numériques calculés.

2. Évaluation et Calcul de Position

Le calcul de la position *y* est déclenché par l'appel de l'objet (ex : f(x)).

Recherche Dichotomique (Spline) : Pour une spline, le calcul de position est local. SciPy effectue une recherche binaire (Binary Search) pour identifier l'intervalle dans lequel se trouve *x*, puis applique l'équation polynomiale locale $ax^2 + bx + c$.

Tableau de synthèse

Concept	SciPy (Numérique)
Nature	Objet de calcul (Coefficients/Mémoire)
Position locale	f(val)
Position globale	f(x_fine)
Avantage	Rapidité extrême, gestion de gros datasets

3. Algorithmes Détaillés

3.1 Évaluation Locale : f(val)

Algorithme : Recherche Dichotomique et Horner Numérique

Lorsque l'objet interp1d est appelé avec une valeur spécifique, il exécute un algorithme en deux phases de complexité globale O(log n).

Phase A : Recherche Dichotomique (Binary Search)

L'objectif est de trouver l'index *i* du segment contenant *x* sans parcourir tout le tableau.

```
Fonction Recherche_Intervalle(x_pts, val):
    gauche = 0
    droite = longueur(x_pts) - 1

    Tant que gauche <= droite:
        milieu = (gauche + droite) // 2

        Si x_pts[milieu] <= val < x_pts[milieu + 1]:
            Retourner milieu  // Intervalle trouvé !

        Sinon Si val < x_pts[milieu]:
            droite = milieu - 1  // Chercher à gauche
```

Phase B : Calcul de la Position Locale

Une fois l'index *i* trouvé, l'algorithme extrait les coefficients de la matrice stockée en mémoire.

```
Fonction Calcul_Position(i, val, matrice_coeffs):
    a = matrice_coeffs[i][0]
    b = matrice_coeffs[i][1]
    c = matrice_coeffs[i][2]

    // Application de la méthode de Horner pour le segment i
    y = c + val * (b + val * a)
    Retourner y
```

3.2 Évaluation Globale : f(x_fine)

Algorithme : Vectorisation et Indexation Avancée (Fancy Indexing)

SciPy applique sa méthode locale, mais massivement parallélisée pour exploiter l'architecture du processeur.

1. **Recherche Vectorisée** : Au lieu de chercher l'intervalle pour un point, la fonction C sous-jacente trouve les index des intervalles pour les 100 points de x_fine en une seule passe optimisée.
2. **Fancy Indexing** : Il crée un tableau contenant tous les coefficients requis pour chaque point de x_fine en un seul accès mémoire.
3. **Broadcasting** : Il effectue la multiplication des polynômes sous la forme $AX^2 + BX + C$ où A, B, C, et X sont de longs vecteurs, utilisant les instructions **SIMD** (Single Instruction, Multiple Data) du processeur pour calculer plusieurs positions à chaque cycle d'horloge.

Pseudo-code : Vectorisation (SIMD)

```
Fonction Evaluer_Vecteur(x_fine, algo_local):
    // 1. Délégation au langage C / Fortran
    // 2. Traitement par blocs (Chunks) dans le cache du
processeur
    // 3. Application de l'algo_local sur plusieurs registres
simultanément
    y_fine = Allouer_Memoire(longueur(x_fine))


    Instruction_Processeur_Vectorielle:
        y_fine[:] = algo_local(x_fine[:])


    Retourner y_fine
```